

Reviews

Book Review: *The C Programming Language* | [Joshua Cochran](#)

Recently I have been making more of an effort to study interesting topics outside of schoolwork. During this, I read *The C Programming Language* by Brian W. Kernighan and Dennis M. Ritchie. Dennis Ritchie is the primary force behind the creation of the C language, and this book is considered the “Bible of C.”

The first half of this book was really fun to read. I had previous exposure to statically typed languages in Java, but I never learned the language deeply. My work experience is centered around data analytics, and as such I have naturally spent most of my time with Python. This being said, this book showed me things I had taken for granted:

- 1. Interactive Shells: When analyzing data, much time is spent in the interactive shell testing ideas and doing general sanity checking; it was very interesting working with a language that needed to be compiled after each change in order to run. However, I noticed that I was a lot more careful with my C code when writing it, and maybe as a result the product was a bit higher quality.
- 2. Extensive Documentation: Python and its popular libraries are documented very thoroughly. Other tools I use frequently like zsh and neovim are also very well documented. I was surprised by how C is maintained, and honestly I still don't understand how to find proper documentation. I found this website, but never anything official. I think I had taken documentation for granted.

Just as I had taken some things for granted using Python, I also learned what I was missing out on by using Python:

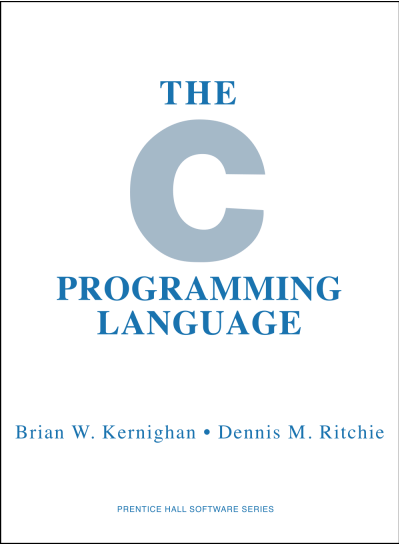
- 1. A Small Language: It is amazing to see how great a small language can be, and makes me wonder if some other languages, like Python, aren't loaded down with unnecessary fluff.
- 2. Static Typing: I learned that I like the straightforward and explicit mannerisms that come with a statically typed language like C.
- 3. Memory Management: I spent most of my time messing around with memory allocation and pointers while learning C. It was so much fun, and I couldn't believe that such intimacy with hardware was actually impossible in Python.

A manual more than a book; I don't mean this in a negative way, it's just fact. The authors make it clear from the beginning that the structure and readability of the book itself, the material reads more like a reference manual and that the book is intended to be concise, and in a lot of ways, that is refreshing. I would rather deal with a book that is less than 250 pages than one that is over a thousand.

It is clear that the authors have a background in mathematics; at times the presentation of the concepts can be confusing, but I would place this as a deficiency on my part more than anything else. Overall, I'm very glad to have read this—I think it's a good read for anyone in computer science who has a curiosity about the structure of programming languages.

Review: *The Codebreaker Reveals a Hidden American Hero* | [Nicholas Norman](#)

(See this review under the Past & Present Section)



CONTINUOUS OBSCURITY

A Computer Science Newspaper

Editor: Nicholas Norman | Presented by the Computer Science Club @ IU Indianapolis

CS Club Visits North Central High School

 | [Nicholas Norman](#)

On March 24th, four valiant members of the Computer Science Club at IU Indianapolis set out to visit a daring place, a high school! North Central High School in northern Indianapolis, near the Nora area.

I was one of the members that went. I am in the middle of the photo, with a ‘flags of the world’ tie on. I really enjoyed visiting the high school, as I am an alumni!

We were invited by the high school’s STEM club, NOBBChE (National Organization for the Professional Advancement of Black Chemists and Chemical Engineers).

Our goal was to expose students to computer science in a non-technical way

and provide experience and guidance on computer science and college in general. As well as the four club members, we were joined by two North Central grads, on the left. They were able to participate in our activity and joined us in a Q&A panel at the end. They were very informative!

The activity highlighted areas such as algorithms (binary search), data structures (trees), and complexity, all without the use of a computer! And, we even got into some areas of game theory and cryptography.

The panel went great. The students were engaged and asked wonderful questions. They probed into areas of college life, as well as the current job market and the effect of AI on modern development. While the students weren’t getting any technical exposure, they were able to glean insights from everyone on the panel.

The CS Club hopes that we can stay in connection with NCHS and go again in the future. Not only was the experience great for the highschoolers, but it gave some speaking experience to our own club members as well.

A Special Thanks To

Port | Dustin Juliano | John Salata | Joshua Cochran

Interested in Submitting a Piece?

Interested in submitting a piece to Continuous Obscurity? Contact the editor at npnorman@iu.edu

Perspectives

Opinion: Preparing for Agentic AI | [Dustin Juliano](#)

The agentic AI paradigm is upon us and is here to stay. Given sufficient time, AI agents will affect the entire global economy in some way or another. There are two primary reasons for this. The first reason is that agentic AI is a development process that scales infinitely with new AI capabilities. Regardless of the term that is eventually used to describe it, the most powerful and capable AI systems can and will be used to create new AI agents.

The second reason agentic AI will have global impact is that there is virtually no aspect of the economy that is not influenced by software. In fact, a large amount of the economy is directly or indirectly augmented or controlled by software systems. Agentic AI, since it is software itself, will go anywhere software can go today, and it will reach even further beyond that in the future through advanced robotics.

We are currently developing AI faster than our digital and physical infrastructure can keep up, and agentic systems are going to accelerate the demand for AI far more than most people realize. As a consequence, it would be wise to discover new means and methods of ensuring that AI systems are as efficient, reliable, and secure as possible, and, ideally, before the agentic AI paradigm reaches its peak. This will require numerous breakthroughs in computer science and artificial intelligence research.

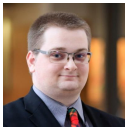
It could very well be the case that AI efficiency, reliability, and security are interdependent, and that the ideal solution will improve all three of these areas simultaneously. Stated conversely, any solution that does not address efficiency, reliability, and security of AI at the same time is unlikely to be globally optimal. This hypothesis, if proven true by one or more solutions, could be helpful in ensuring economic and societal stability as we continue to integrate with AI unabated.

Our Authors

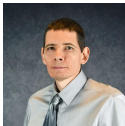
Nicholas Norman is a computer science and game development student at IU Indianapolis. He is a teaching assistant and research assistant. Check out his website for projects and his personal blog at <https://npnorman.github.io/>



John Salata is a computer science student at IU Indianapolis with a major in computer science and a specialization in cognitive automation. For more information, see his personal website and blog at <https://salata.software/>



Dustin Juliano is a computer science student, author, and researcher. He is interested in areas of formal artificial intelligence and AI security. Find more information about his publications and his personal blog, see <https://dustinjuliano.com/>



Joshua Cochran is a masters student at IU Indianapolis studying computer science and a research assistant for the IU School of Medicine. He has a background in psychology and data science. Check out his website at <https://venvio.blog>



Disclaimer

The views held by individual authors may not be reflected by Continuous Obscurity, the editor, or other authors.

Theory & Practice

The Importance of Software Testing & Build Tools | [John Salata](#)

One of the easiest ways to make a software project unmaintainable and poorly documented is by not using appropriate build tools. Build tools simplify the process to compile, test, and package your code. Some languages and frameworks integrate these tools with the language. Node projects, for example, will set everything up for you automatically with a simple npm init. Other languages like Python and Java have multiple build tools available that require the developer to make an intentional choice on which system to use. If your project has any level of complexity, using these tools should not be optional.

Why? Most of these tools make it extremely easy to test your code.

Software testing, like writing documentation, can be tedious and often forgotten about. However, it can make or break the end result.

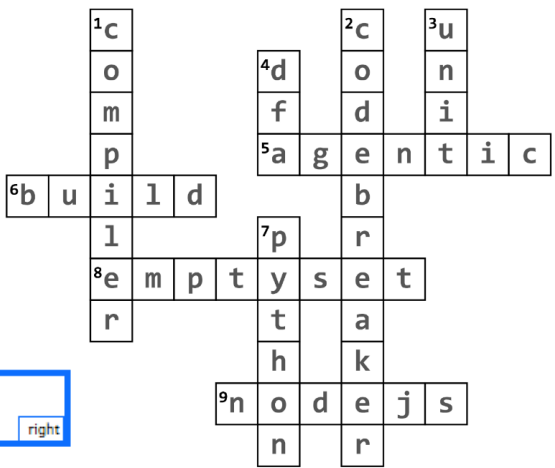
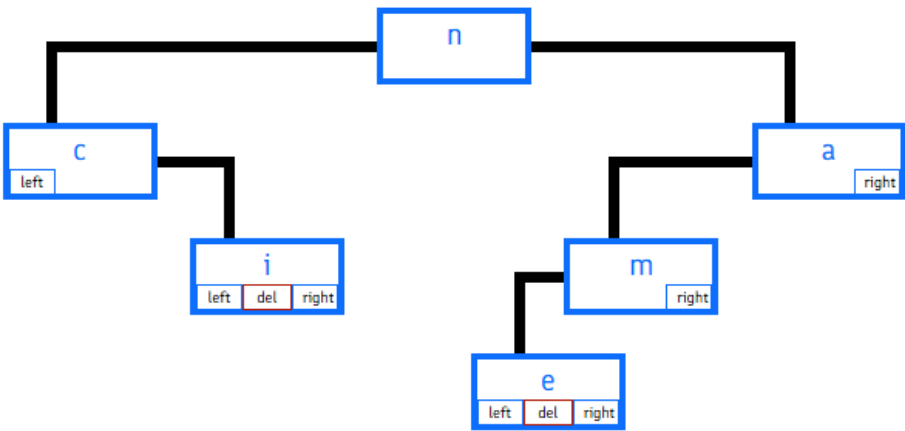
There are numerous testing frameworks out there for each language and stack that you might be using. Start small by testing each individual class or function you are working on. Make sure that expected inputs work properly and that any invalid inputs are handled gracefully. These unit tests go a long way to making sure that everything else works. For example, if you make a change that will break something else, you can catch it with your tests and have an easier time identifying the fix.

There is much more that can be said about software testing and its importance. But the best way to learn is by doing.

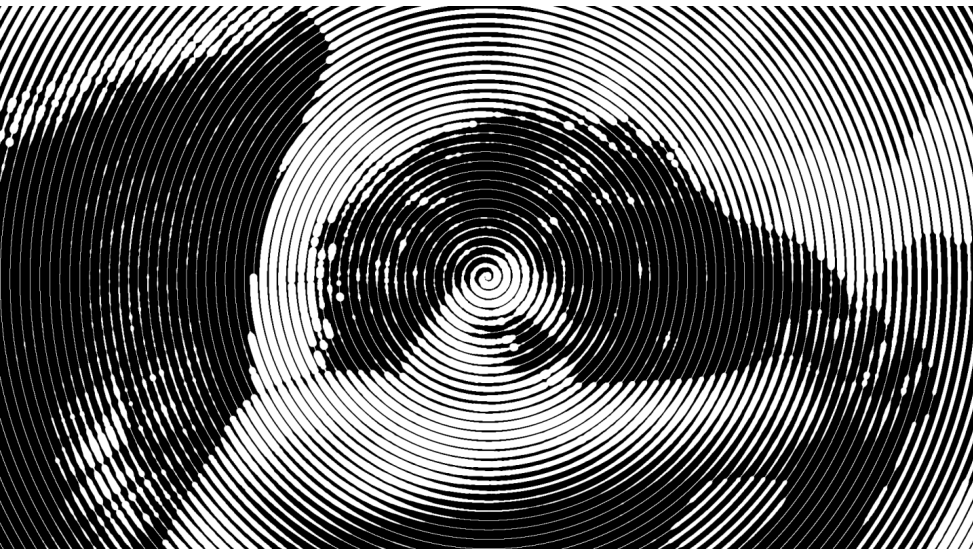
Answers to Puzzles

Puzzle Answer

The answer is that he works at the local movie theater, or the ‘cinema.’ We can build a binary tree like this (figure below), then the inorder traversal is ‘cinema’ and the postorder traversal is ‘iceman.’ There is an algorithm to build a binary tree from two traversals, so it may be possible to find the anagram first and build the tree to verify, or to try different tree combinations to find the anagram. As for the theater, I did go in the coming weeks, but they said Dr. Loops got a new job again, that of which I do not know. Maybe I’ll see him again soon.



Art & Culture



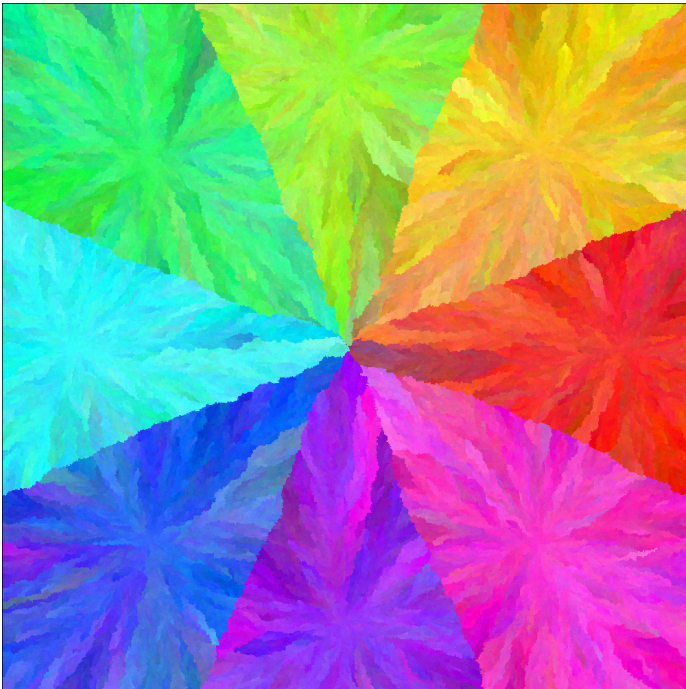
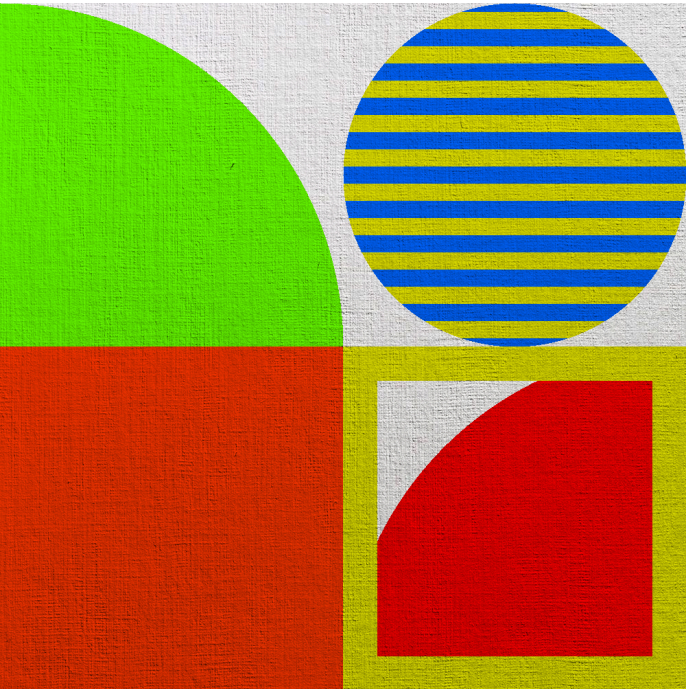
Computer Science isn’t all technical jargon and codes. Here are three pieces of art created by Port, an ex-student of Indiana University Indianapolis (formerly IUPUI). No generative AI was used in the making of these pieces.

This piece, a rendering of The Fallen Angel by Alexandre Cabanel, is a continuous spiral of various thickness, reminiscent of Claude Mellan’s Sudarium of St. Veronica. The ever-changing stroke weight is directly controlled by the brightness of the underlying image.

The piece to the lower right is the result of a cellular automaton seeded by points around a color wheel. The colored cells spread to neighboring blank cells, slightly mutating their color each time. This algorithm is commonly referred to as Huegene, a portmanteau of hues + genes.

The piece to the lower left is an abstract, geometric rendering in the Bauhaus design style. It consists of a paper texture overlaid with a procedurally-generated grid of simple shapes and patterns in vibrant colors.

For any inquiries, please reach out to portalsrule@gmail.com.



Past & Present

The First Compiler was “Stupid” | [Nicholas Norman](#)

Grace Hopper’s Compiler was “very stupid,” according to her own words from the “Oral History of Captain Grace Hopper,” archived by the Computer History Museum. But a simple or “stupid” idea that may seem to come about from common sense, can have massive impacts, as we see in modern day compilers.



According to her biography from Yale University, Grace Hopper was a naval officer and computer pioneer of the 20th century. Originally a math professor at Vassar College, she joined the navy and worked on MARK I, a general-purpose computer. Here is where the foundations of the first compiler were laid. She would later work on languages such as FLOW-MATIC and its successor, COBOL.

There were problems with coding MARK I. As it was one of the first computers, there were bound to be. She notes that academics and mathematicians wanted to code with symbols and others wanted words to code with. They only had numerical codes. Along with this problem, there was no formal idea of programming. If you wanted to code, you had to write the codes, or copy them by hand from someone else’s code book. It was tedious and error-prone.

Hopper had the wit to recognize the problems laid before her and drew inspiration from Betty Holberton, who created one of the first programs to generate a program. She pushed the limits of what a computer was thought capable of by collecting common codes, represented by words and not numbers, and made the A-O compiler.

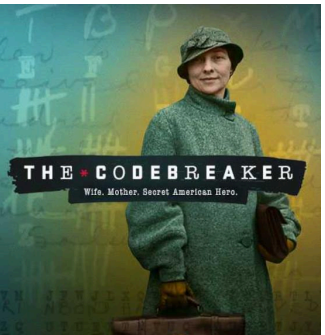
It was a collection of subroutines. It also loaded and linked the programs, which was done by hand before. “It was very stupid.” That’s how she describes that first compiler. But it was common sense and her disdain for “doing anything over and over again” that led to her invention.

It is interesting how such a simple idea can produce massive results. The idea that a computer could translate words into its own language was revolutionary even though it may not seem like it since every modern programming language has a compiler which does so. It took an understanding of differing perspectives and the courage to not underestimate a computer’s capabilities to give us a foundation of modern programming languages.

Review: *The Codebreaker* Reveals a Hidden American Hero | [Nicholas Norman](#)

Documentary | 53 min | 2023 | American Experience, PBS

The Codebreaker follows the life of Elizebeth Friedman, a pioneer in modern cryptanalysis, from decoding Shakespeare to breaking codes from the mafia and the Nazis. It follows her struggles with her family, her work, her husband, and the patriarchal world around her. But even so, Friedman used her skills to help others and wasn’t afraid to take action to do what was right.



It’s a great introduction to cryptography and cryptanalysis, describing it in simple and understandable terms. It explores many relevant events to modern history, through prohibition, WWII, and even the foundations of the NSA. Even though Friedman never used a computer to crack codes, her influence is still seen in modern cryptography to this day. I recommend it wholeheartedly!

The documentary can be found on Amazon Prime or for free by searching “Film Screening and Discussion of ‘The Codebreaker’” on YouTube, by the channel HMTTC, skipping to timestamp 9:58.

Puzzles

Crossword | [Nicholas Norman](#)

Across

- 5. AI paradigm that can run autonomously
- 6. _____ Tools, for automating creation of applications
- 8. The language of a dfa that rejects every word
- 9. Backend JS environment

Down

- 1. What translates high-level code to assembly
- 2. Title of Elizebeth Friedman
- 3. A type of testing, individually
- 4. Deterministic Finite Automaton
- 7. Programming language named after comedy group

Dr. F. W. Loops at the Gas Station

[Nicholas Norman](#)

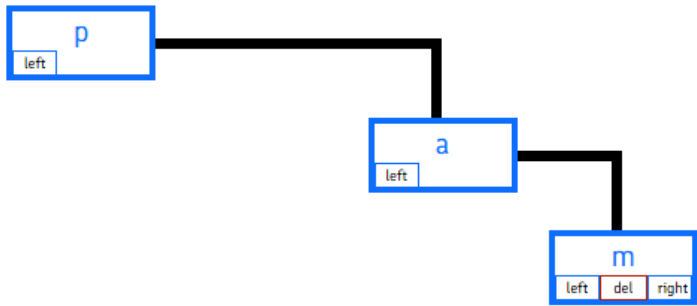
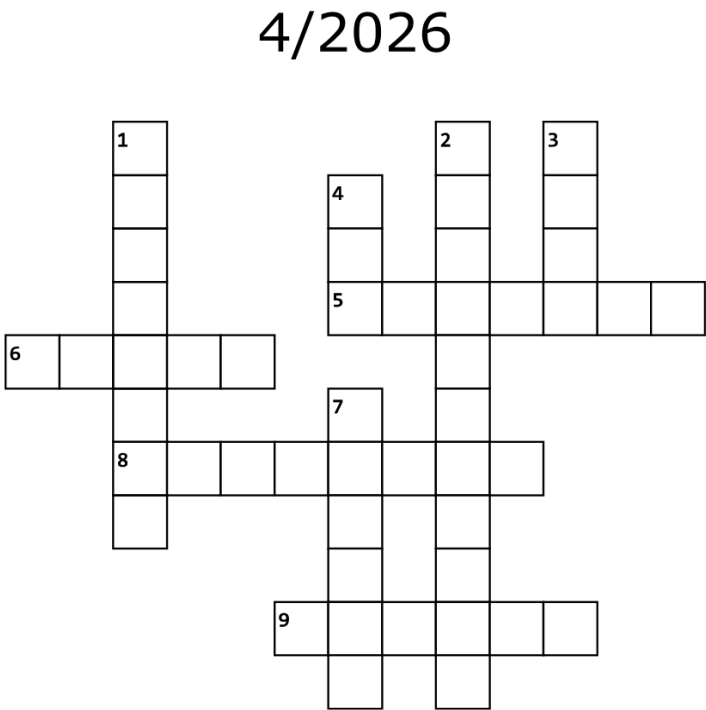
While working the farmers market on an early Saturday morning, me and my boss stopped by a gas station to pick up our weekly share of ice bags for the coolers. An ice truck was blocking our way, so I had to wait for the iceman to take the fresh bags out. To my surprise, the man that took the bags out was one of my retired professors, Dr. Fredrick W. Loops! He noticed me and started talking. He talked about the trees on his route and the traffic. I asked him why he decided to become an iceman after retiring and he said it reminded him of hauling ice blocks with his father when he was a kid.

My boss was getting annoyed. I had to grab the bags and wrap up the conversation, so I asked Dr. Loops for his contact, but in his stubborn way, he didn’t answer straight. He told me to look for him at his new job. Hauling ice was boring. He said his new place of business was an anagram of his current position. Of ‘iceman.’ But, he said it had a condition. It was also a postorder traversal anagram. I was very confused.

He pulled out a map and said “This is how I remember my daughter Pam. If I draw a binary tree like so, (figure right) then an inorder traversal would be ‘pam’ and a postorder traversal would be ‘map.’ Pam and map!”

He said “A postorder traversal anagram means if each letter was put in a binary tree and the inorder and postorder traversals were read, the words would be anagrams of each other.”

So reader, can you find the place of work I should be searching for to reconnect with Dr. F. W. Loops? Make sure to verify that it is a postorder traversal anagram. I don’t want to meet him in the wrong place!



Humor

Humor: Regular Languages Aren’t So Regular | [Nicholas Norman](#)

Studying the theory of computation and formal languages, you learn about all types of languages. A language is just a set of words. One family of languages you’ll learn about is the regular languages. You may be familiar with one way to accept these types of languages, regular expressions (regex). Although there are other ways to accept them as well.

But how “regular” can these languages be? Let’s use some regular expressions to see.

Given some alphabet, like {a,b}, we can form a regular language represented by something like a*b. This will accept words with 0 or more a’s and exactly one b. The * means 0 or more. Words like “ab,” “aaaaaaaab,” and “b”, are all a part of this language. Words like “ba” and “a” are not. And since we can represent this language using a regular expression, it is a regular language.

So what about a*? Yep, that’s regular too. Words like a, aaaa, and λ, are all a part of this language. Lambda?! Where did that come from? λ (lambda) represents the empty string, such as “” with no characters in between, not even spaces. a* means 0 or more a’s, so 0 a’s leads to λ. What’s so regular about the empty string! That’s like someone walking up to you, opening their mouth, and calling that word! What about the opposite side of the spectrum, with words like aaa...a, that have a huge number of a’s. Can there be 1 million a’s in a word? Sure, that’s in the language! 1 trillion? Even better! 1 googol? Why not! If it takes a tenth of a second to say one “a,” then a word with 1 million a’s would take over 27 hours to say. That’s *very* “regular,” mhm.

If we can reject words and accept words, are there regular languages that accept every word? With our alphabet {a,b}, something like (a|b)* would do the trick. We can choose an a or b (|), then repeat the choice (*). Words like “ababab,” λ, “bbba,” and any other word is accepted. How is a language, where every combination of letters is valid, regular?!

So we have a language that accepts every word. What about the opposite? A language that rejects every word. That’s surely irregular right? What we are describing is the complement of the language (a|b)*. The complement is the set of all the words not accepted into some language. So, if the language accepts every word, then the complement is... no words! It’s an empty set. And it can be proven that the complement of a regular language is also regular! How is a language with no words “regular?” That’s not even opening your mouth. That’s just staring at someone.

As you can see, there are many weird quirks and edge cases that make these languages feel very irregular, but even so, these languages are very powerful and we use them all the time. I just want to know who came up with this term “regular,” because it doesn’t seem to fit!

Eat One Rock Per Day

By Nicholas Norman

I had trouble programming quads,
Search google but making no odds,
The server sent packets,
A fix for my graphics,
But the overview was a fraud.

Jokes | [Nicholas Norman](#)

Why was Anakin Skywalker destined for evil?
Because he used Force Push.
Not sure when you’ll get it.
Want to hear a joke about race conditions?

Interested in submitting a piece to Continuous Obscurity? Contact the editor at npnorman@iu.edu